Secure Network Protocols
HTTPS - SSL\TLS

Dr. Muhanned
Department of CSE
2nd Semester
Undergraduate

When the Problem Started

- As the World Wide Web became popular and commercial enterprises began to take an interest in it, it became clear that some level of security would be necessary for transactions on the Web.
- Example of this is making purchases by credit card.
- First, you might worry that the information would be intercepted in transit and subsequently used to make unauthorized purchases. You might also worry about the details of a transaction being modified, for example, to change the purchase amount.
- Second, you would certainly like to know that the computer to which you are sending your credit card information is in fact one belonging to the vendor in question and not some other party.
- Thus, we immediately see a need for confidentiality, integrity, and authentication in Web transactions.
- The first widely used solution to this problem was SSL, originally developed by Netscape and subsequently the **basis** for the IETF's TLS **standard**.

HTTPS - SSL\ TSL

- The designers of SSL and TLS recognized that these problems were not specific to Web transactions (i.e., those using HTTP) and instead built a general-purpose protocol that sits between an application protocol such as HTTP and a transport protocol such as TCP.
- The reason for calling this “transport layer security” is that, from the application’s perspective, this protocol layer looks just like a normal transport protocol except for the fact that it is secure.
- That is, the sender can open connections and deliver bytes for transmission, and the secure transport layer will get them to the receiver with the necessary confidentiality, integrity, and authentication.
- By running the secure transport layer on top of TCP, all the features of TCP (reliability, flow control, congestion control, etc.) are also provided to the application. This arrangement of protocol layers is depicted in Figure 8.15.

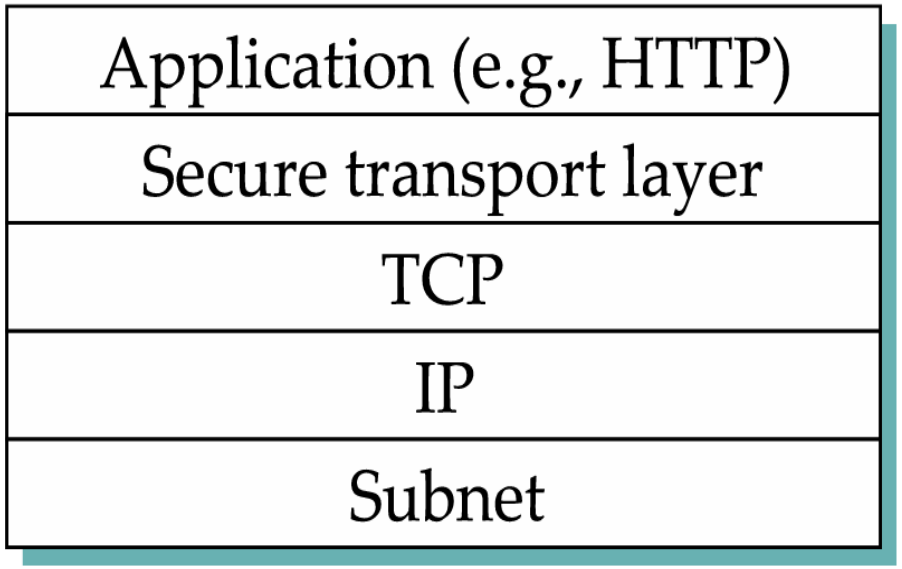


Figure 8.15 Secure transport layer inserted between application and TCP layers.

HTTPS - SSL\ TSL

- When HTTP is used in this way, it is known as HTTPS (Secure HTTP). In fact, HTTP itself is unchanged.
- It simply delivers data to and accepts data from the SSL/TLS layer rather than TCP.
- For convenience, a default TCP port has been assigned to HTTPS, port number 443.
- That is, if a node try to connect to a server on TCP port 443, the node will likely be talking to the SSL/TLS protocol, which will pass the node's data through to HTTP provided all goes well with authentication and decryption.
- Although stand-alone implementations of SSL/TLS are available, it is more common for an implementation to be bundled with applications that need it, primarily web browsers.

Handshake Protocol

- A pair of TLS participants negotiate at runtime which cryptography to use. The participants negotiate a choice of:
  1. Data integrity hash, MD5 or SHA, used to implement HMACs.
  2. Symmetric-key cipher for confidentiality. Among the possibilities are DES, 3DES, and AES.
  3. Session-key establishment approach. Among the possibilities are Diffie-Hellman, Fixed Diffie-Hellman. Also, public-key authentication protocols, such as RSA or DSS, are used to authenticate session key parameters.
- Interestingly, the participants may also negotiate the use of a compression algorithm, not because this offers any security benefits, but because it's easy to do while negotiating all integrity, confidentiality, and session key, to do some expensive per-byte operations on the data.

Handshake Protocol

- In TLS, the confidentiality cipher uses two keys, one for each direction, and similarly two initialization vectors.
- The HMACs are likewise keyed with different keys for the two participants.
- Thus, regardless of the choice of cipher and hash, a TLS session requires six keys.
- TLS derives all of them from a single shared master secret. The master secret is a 384-bit (48-byte) value that is in turn derived in part from the session key that results from TLS's session-key establishment protocol.
- The part of TLS that negotiates the choices and establishes the shared master secret is called the handshake protocol. (Actual data transfer is performed by TLS's record protocol.)
- The handshake protocol is at heart a session-key establishment protocol, with a master secret instead of a session key.

Handshake Protocol

- TLS supports a choice of approach to session-key establishment, ranging from public-key certificates to Diffie-Hellman.
- Furthermore, the handshake protocol supports a choice between mutual authentication of both participants, authentication of just one participant (this is the most common case; e.g., authenticate a website but not a user), or no authentication at all (anonymous Diffie-Hellman).
- Thus, the handshake protocol knits together several session key establishment protocols into a single protocol.
- The workflow of TLS handshake protocol is shown in Figure 8.16.
- The workflow of TLS is as follows:
  1. The client initially sends a list of the combinations of cryptographic algorithms that it supports, in decreasing order of preference and its Nonce (N_c).

Handshake Protocol

2. The server responds giving the single combination of cryptographic algorithms it selected from those listed by the client. These messages also contain a client-nonce and a server-nonce (N_s), respectively, that will be incorporated in generating the master secret later.
3. At this point, the negotiation phase is complete. The server now sends additional messages based on the negotiated session key establishment protocol (one of the possibilities is anonymous Diffie-Hellman, so it wouldn't be accurate to call it an authentication protocol). That could involve sending a public-key certificate or a set of Diffie-Hellman parameters.
4. If the server requires authentication of the client, it sends a separate message indicating that. The client then responds with its part of the negotiated key exchange protocol.

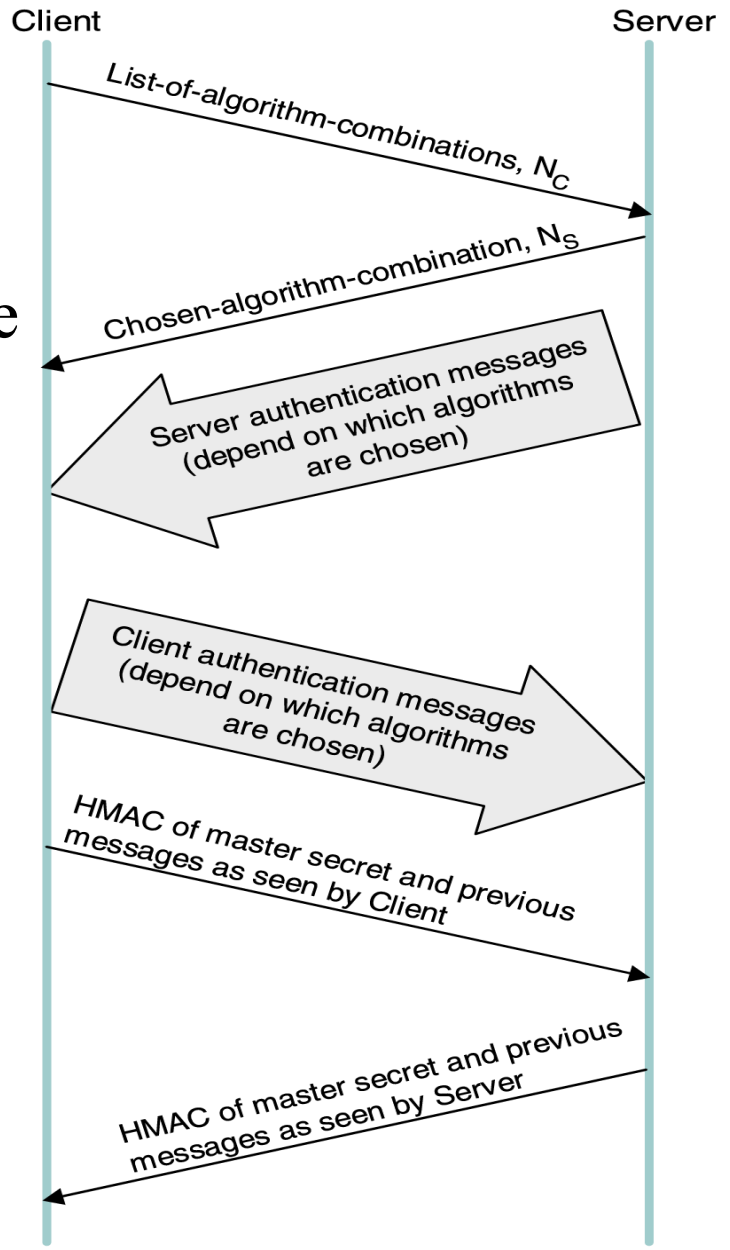
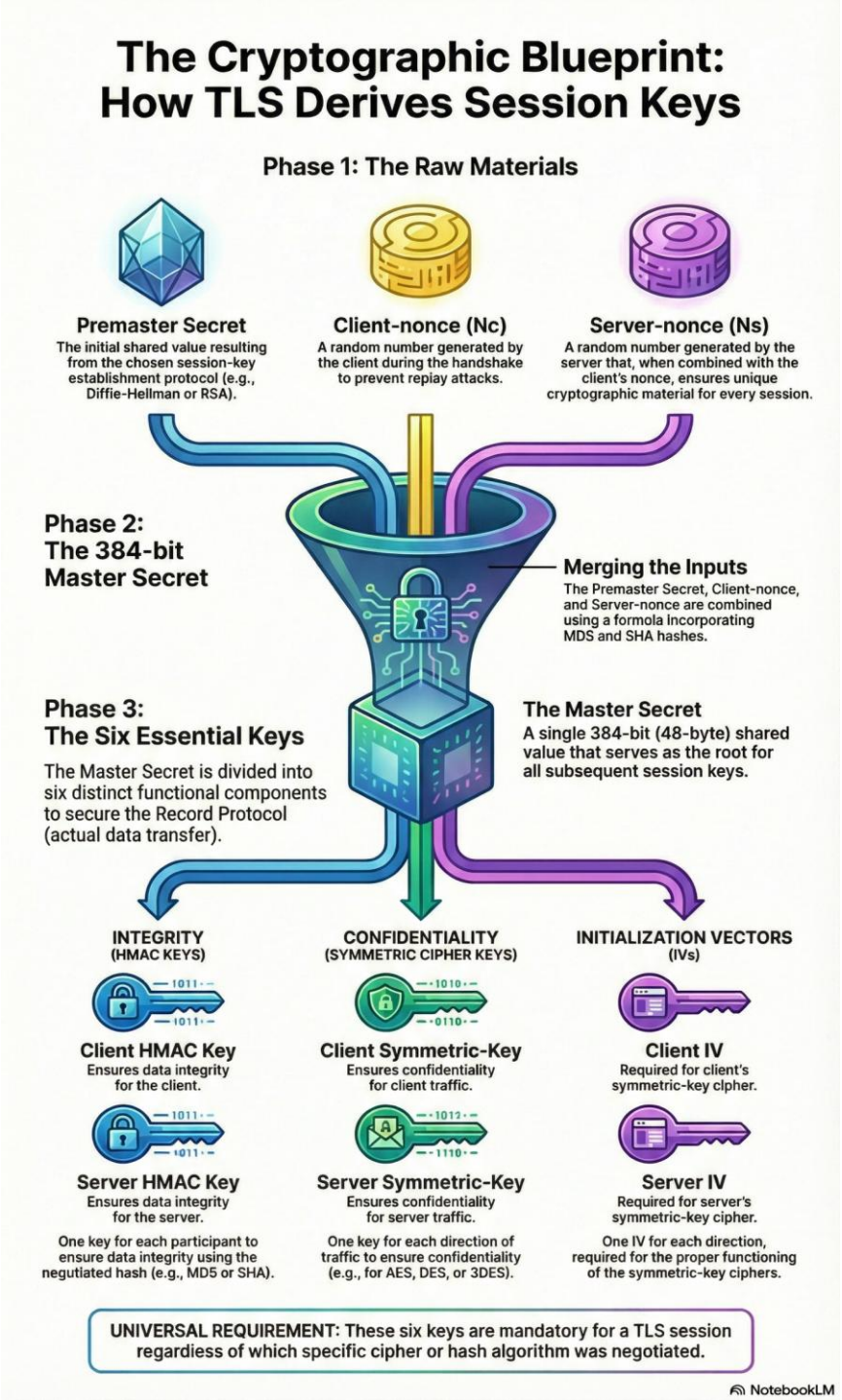


Figure 8.16 Handshake protocol to establish TLS session.

Handshake Protocol

- Now the client and server each have the information necessary to generate the master secret. The session key that they exchanged is not in fact a key, but instead what TLS calls a premaster secret. The master secret is computed (using a published formula incorporating both MD5 and SHA) from this premaster secret, the client-nonce, and the server-nonce.
  - **Client-nonce**: A random number generated by the client during the handshake.
  - **Server-nonce**: A random number generated by the server during the handshake.
  - **Purpose**: These nonces ensure that each session has unique cryptographic material, even if the same keys are reused. They prevent replay attacks (where an attacker tries to reuse old handshake data).
- Using the keys derived from the master secret, the client then sends a message that includes a hash of all the preceding handshake messages, to which the server responds with a similar message. This enables them to detect any discrepancies between the handshake messages they sent and received, such as would result, for example, if a man-in-the-middle modified the initial unencrypted client message to weaken its choices of cryptographic algorithms.

Handshake Protocol



Record Protocol

- Within a session established by the handshake protocol, TLS's record protocol adds confidentiality and integrity to the underlying transport service. Messages handed down from the application layer are:
  1. Fragmented or coalesced into blocks of a convenient size for the following steps;
  2. Optionally compressed;
  3. Integrity-protected using an HMAC;
  4. Encrypted using a symmetric-key cipher;
  5. Passed to the transport layer (normally TCP) for transmission.
- The record protocol uses an HMAC as an authenticator. The HMAC uses MD5 or SHA-1, whichever was negotiated by the participants. The client and server have different keys to use when computing HMACs, making them even harder to break. Furthermore, each record protocol message is assigned a sequence number, which is included when the HMAC is computed—even though the sequence number is never explicit in the message. This implicit sequence number prevents replays or reordering's of messages.

Record Protocol

- This is needed because, although TCP guarantees sequential no-duplicate messages under normal assumptions, those assumptions do not include an adversary that can intercept TCP messages and send bogus ones. On the other hand, it is TCP's delivery guarantees that make it possible for TLS to rely on a legitimate TLS message having the next implicit sequence number in order.
- Another interesting feature of the TLS protocol, which is quite a useful feature for Web transactions, is the ability to "resume" a session. To understand the motivation for this, it is helpful to understand how HTTP makes use of TCP connections.
- Each HTTP operation, such as getting a page of text or an image from a server, requires a new TCP connection to be opened. Retrieving a single page with a number of embedded graphical objects might take many TCP connections.

Record Protocol

- Once the TCP connection is ready to accept data, the client would then need to start the TLS handshake protocol, taking at least another two RTTs (and consuming some amount of processing resources and network bandwidth) before actual application data could be sent. The resumption capability of TLS alleviates this problem.
- Session resumption is an optimization of the handshake that can be used in those cases where the client and the server have already established some shared state in the past. The client simply includes the session ID from a previously established session in its initial handshake message. If the server finds that it still has state for that session, and the resumption option was negotiated when that session was originally created, then the server can reply to the client with an indication of success, and data transmission can begin using the algorithms and parameters previously negotiated. If the session ID does not match any session state cached at the server, or if resumption was not allowed for the session, then the server will fall back to the normal handshake process.